High-Level System Design for a Document System

1. First, understand what I’m designing

Our application is a lightweight version of Google Docs.

The assessment requires users to create documents, edit formatted text, upload files, share documents with other users, and reopen their work later.

There are two different concepts here:

• Document sharing: Alice gives Bob permission to open or edit her document.

• Real-time collaboration: Alice and Bob type simultaneously and immediately see each other's changes.

The first is required. The second is optional.

That distinction is critical because real-time collaboration introduces an entirely different level of architectural complexity.

Functional requirements

Feature	What the system must do
Document management	Create, rename, open and delete documents
Rich-text editing	Bold, italic, underline, headings and lists
Persistence	Save content and preserve formatting
File import	Convert an uploaded file into an editable document
Sharing	Owner grants access to another user
Access control	Prevent unauthorized viewing and editing
Deployment	Provide a working public URL

Nonfunctional requirements

In a system design interview, we'd also discuss performance, reliability, security, scalability and cost.

For this assignment, my priorities are different from those of a production-scale Google Docs application.

Priority	Level
Correctness and working features	Highest

Priority	Level
Deployment reliability	Highest
Security of document access	Important
Low implementation complexity	Important
Massive scalability	Not required
Real-time multiuser editing	Optional
System design principle: I should not optimize for millions of concurrent users when the immediate requirement is a reliable, demonstrable application for reviewers.	# 2. My currently selected architecture
This is the design I selected during planning. It is a proposed architecture, not yet a verified running implementation.	Browser — React + TipTap
Document list · Rich-text editor · Sharing UI · File import	HTTPS / JSON / Session cookie
Next.js application on Vercel	API Route Handlers — TypeScript
Authentication	Who is making the request?
Validation (Zod)	Is the input valid?
Authorization	Can this user perform the action?
Business logic	Create, save, share or import
Prisma ORM	Database access and migrations
SQL queries	PostgreSQL on Neon
Users · Documents · Document shares	Persistent relational database
This is called a modular monolith: one application deployment, with separate modules responsible for authentication, document operations, permissions, imports and database access.	I do not need a microservices architecture to achieve good separation of responsibilities.
# 3. Architecture decision 1: Monolith vs separate backend vs microservices	This is the first major system design decision.
Option A — Modular monolith	Selected
Next.js	React UI + REST API
Postgres	One application deployment, organized into modules.
Option B — Separate frontend/backend	React

Priority	Level
FastAPI / Express	Postgres
Two separately deployed applications.	Option C — Microservices
Auth	Docs
Sharing	Files
Data stores	Independent services and deployment lifecycles.
| Consideration | Monolith | Separate API | Microservices |	| ---------- | --------- | ---------- | ---------- |
| Initial development | Fast | Moderate | Slow |	| Deployment | Simple | Two deployments | Many deployments |
| Independent scaling | Limited | UI/API separated | Per service |	| Debugging | Simplest | Moderate | Distributed tracing needed |
| Operational overhead | Low | Moderate | High |	| Fit for this assignment | Excellent | Good | Poor |
### Why I’m choosing the monolith	Imagine Alice opens a document. The application must identify Alice, check whether she owns the document, retrieve its content and return it.
With a modular monolith, all of that can happen within one server-side request handler, using local function calls and a database query.	With microservices, we might need separate network calls to an authentication service, permission service and document service. That creates more latency, more failure points and more deployment complexity without solving an actual problem at my current scale.
### What I’m sacrificing	I cannot independently deploy or scale the document API and sharing API when they live in the same application deployment.
However, a monolith is not automatically unscalable. Multiple instances of a stateless monolith can handle traffic in parallel. I can also optimize database queries, add caching and scale database capacity before needing microservices.	When would we reconsider? When different modules have substantially different runtime, scaling, reliability or team-ownership requirements. For example, a persistent WebSocket collaboration service would be a reasonable candidate to separate from the request-response API.
# 4. Architecture decision 2: Why Next.js instead of Python FastAPI?	You previously suggested Python, which is a legitimate alternative. We chose Next.js primarily because of the timebox, not because Python is technically inferior.
| Factor | Next.js + TypeScript | React + FastAPI |	| ---------- | ---------- | ---------- |
| Frontend | React | React |	| Backend | Next.js route handlers | Python FastAPI |
| Languages | TypeScript | TypeScript + Python |	| API validation | Zod | Pydantic |

Priority	Level
| Database access | Prisma | SQLAlchemy |	| API documentation | Manual/additional tooling | Built-in OpenAPI |
| Deployment | One application | Can be one or two deployments |	| Best advantage | Faster integrated development | Strong standalone Python API |
Why Next.js wins for this assignment: It minimizes integration work and deployment setup. I can use TypeScript across UI, API handlers and shared data contracts.	What we give up: FastAPI's excellent built-in API documentation and a Python backend that may be preferable for Python-heavy workloads.
Also, using Next.js does not mean I am giving up backend engineering. Authentication, authorization, persistence, validation and server-side business logic are still backend responsibilities.	# 5. Architecture decision 3: Why PostgreSQL instead of MongoDB, SQLite or Firebase?
My data has natural relationships.	User
id (PK) · email · name	owns documents / receives shares
Document	id (PK)
ownerId (FK)	title
content (JSONB)	updatedAt
DocumentShare	id (PK)
documentId (FK)	userId (FK)
role	createdAt
PK = primary key; FK = foreign key. DocumentShare connects users to documents and represents a many-to-many access relationship.	Suppose Alice owns document 101 and shares it with Bob and Carol. We store one document row and two share rows. I do not duplicate the document's content for each user.
| Database | Advantage | Tradeoff |	| ---------- | ---------- | ---------- |
| PostgreSQL | Foreign keys, transactions, JSONB, relational queries | Requires hosted database management |	| SQLite | Lightweight, excellent local development | File-based storage unsuitable for Vercel's ephemeral runtime without a persistent external service |
| MongoDB | Flexible document structures | Relationships and constraints require more application-level design |	| Firebase Firestore | Managed scaling, real-time listeners | Different query and security-rule model; more vendor-specific |
| Cloudflare D1 | Managed SQLite at the edge | Cloudflare-specific integration and deployment choices |	My choice: PostgreSQL on Neon. I need durable persistence, user-document relationships and simple constraints.

Priority	Level
For example, UNIQUE(documentId, userId) prevents Alice from accidentally sharing the same document twice with Bob.	### Why Prisma?
Prisma sits between our API and PostgreSQL. It lets us define a schema, generate typed database queries and manage migrations.	We could use raw SQL or Drizzle instead. Raw SQL provides greater control; Drizzle is lighter and more SQL-oriented. Prisma trades some runtime and tooling overhead for a convenient developer workflow.
One important distinction: Prisma is not the database. PostgreSQL stores the data; Prisma is the application-side database access layer.	# 6. Architecture decision 4: How do we save documents?
This is where system design becomes particularly interesting.	Consider Alice typing 100 characters.
We have several choices:	| Strategy | Advantage | Disadvantage |
| ---------- | ---------- | ---------- |	| Save every keystroke | Very frequent persistence | Excessive requests and database writes |
| Manual Save button | Simple | Users may forget to save |	| Debounced autosave | Fewer writes, convenient UX | Recent unsaved edits can be lost |
| Real-time operation synchronization | Supports simultaneous editing | Much more complex infrastructure |	We selected debounced autosave, approximately one second after the user stops typing.
Example: Alice types a sentence	Illustrative timeline: each new edit resets the debounce timer. A save begins after a quiet period.
The API receives the document's structured JSON and persists it in PostgreSQL.	But there's a problem.
Suppose Alice and Bob open the same document at the same time. Alice changes the heading, while Bob changes a paragraph. Both started with the same old content.	If Bob's full-document save reaches the database after Alice's, Bob's version could overwrite Alice's heading change.
This is the lost update problem.	Our initial plan accepts last-write-wins behavior because simultaneous co-editing is not required. I should explicitly document that limitation.
A relatively inexpensive improvement is optimistic concurrency control:	1. Add a document version number.
2. When Alice loads the document, she receives version 5.	3. When she saves, she submits the expected version 5.
4. The database updates only if the current version is still 5.	5. If Bob already saved version 6, Alice receives a conflict instead of silently overwriting Bob.
That prevents silent overwrites, but we would still need a user experience for resolving conflicts. True simultaneous editing would eventually call for something like Yjs and a real-time collaboration service.	# 7. Architecture decision 5: Authentication vs authorization

Priority	Level
These are different problems.	Authentication: Who are you?
Authorization: What are you allowed to do?	Our proposed implementation uses three seeded users — Alice, Bob and Carol — with a user picker and an HMAC-signed session cookie.
This is permitted by the assessment, but it is not production authentication. Anyone using the demo can select any seeded identity.	For authorization, we check ownership and sharing permissions on the server.
| Operation | Owner | Editor | Viewer | Outsider |	| --- | ----- | ------ | ------ | -------- |
| Read document | Allow | Allow | Allow | Deny |	| Edit content | Allow | Allow | Deny | Deny |
| Rename | Allow | Allow | Deny | Deny |	| Share document | Allow | Deny | Deny | Deny |
| Delete document | Allow | Deny | Deny | Deny |	Why not use OAuth or Auth.js immediately? They would provide real identity verification, but introduce configuration and testing work unrelated to the assignment's core editor functionality.
The important engineering rule is that hiding an Edit button is not authorization. Every API mutation must independently verify permissions.	We also need server-side validation, safe rendering of imported content, secure cookie settings and protection against cross-site request forgery where applicable.
# 8. Architecture decision 6: File import vs object storage	The assignment requires a file upload workflow, but it does not require storing original files.
We have two primary options.	Option A: Import into an editable document
Selected	Upload
Convert	Save JSON
The original file is processed and converted into editable document content. No separate file-storage service is necessary.	Option B: Preserve original files in object storage
Upload	S3 / R2
Metadata	The file remains available for download, auditing or reprocessing, but requires additional storage and access-control logic.
For my application, import is the simpler and more useful product experience.	The initial plan supports .txt, .md and .docx. Conversion should use established parsing libraries and a controlled TipTap schema. We must still validate file size, type and resulting content; conversion alone is not a security guarantee.
If we later support attachments, images or original-file downloads, I can introduce S3 or Cloudflare R2.	# 9. Architecture decision 7: Hosting

Priority	Level
The previous design selected Vercel for the Next.js application and Neon for PostgreSQL.	Why not Cloudflare, AWS or Render?
| Option | Main benefit | Main tradeoff |	| ---------- | ---------- | ---------- |
| Vercel + Neon | Straightforward Next.js deployment | Two hosted services; serverless runtime constraints |	| Cloudflare Workers + D1 | Integrated edge hosting and database | Runtime compatibility and deployment changes |
| Render | Conventional application-server hosting | Free-instance availability and cold-start considerations |	| AWS ECS/EC2 + RDS | More infrastructure control | More setup, operations and potential cost |
I am choosing Vercel because deployment simplicity is more valuable than infrastructure flexibility for this assessment.	I should verify current free-tier quotas and framework compatibility during deployment rather than rely on historical numbers.
# 10. How the entire system works, step by step	Let's trace one complete user flow.
1. Alice selects her demo account.	The session endpoint sets a signed cookie. Subsequent API requests use the cookie to identify the selected user.
2. Alice creates a document.	React sends POST /api/documents. The backend validates the request, obtains Alice's user ID and inserts a document with ownerId = Alice.
3. Alice writes and formats content.	TipTap maintains structured document content in the browser. Bold, headings and lists become nodes or marks in its JSON representation.
4. The application autosaves.	After the debounce interval, the browser sends PATCH /api/documents/:id. The backend checks edit permission and updates the database.
5. Alice shares with Bob.	The backend verifies that Alice owns the document and inserts a DocumentShare row for Bob.
6. Bob opens Shared with me.	The backend queries documents where Bob has a share record. Bob can open the document and edit it only if his assigned role permits editing.
7. Carol attempts to open the same document.	The backend finds neither ownership nor a share record and refuses access. The UI cannot override this decision.